



《OpenCV 基础与应用》

实验报告

实验序号： 实验 2

实验名称： 医学影像切片显示与窗宽窗位调整

姓名： 何宗霖

学号： 23374275

同组： 无

专业： 飞行器控制与信息工程

2025 年 11 月 1 日

一、实验目的：

掌握图像矩阵像素访问方法及超 8 位深度图像的窗宽窗位调节显示。

二、实验内容：

1、利用 OpenCV 库设计并实现一软件医学图像观察软件，要求：

【如果 C++ 编程】：

- ① 读入 CT 图像序列，显示过参考点的轴截面、矢状面和冠状面图像，在轴截面图像上显示图像立方体灰度的最大值与最小值；
- ② 可用滑动条修改参考点空间位置，实现浏览所有位置的截面图像浏览；
- ③ 可用滑动条调整窗宽窗位以调节显示；

【如果 python 编程】（未选择）：

- ① 读入 IVUS 超声图像序列，显示过参考点的实现横断面与过中心轴线任意切面图像的显示，在轴截面图像上显示图像立方体灰度的最大值与最小值；
- ② 可用滑动条修改参考点空间位置和切面角度，实现所有位置截面图像浏览；
- ③ 可用滑动条调整窗宽窗位以调节显示；

2. 【挑战-选做】利用鼠标在图像上结合键盘选择参考点和切面位置(或切面角度)，以及调整窗宽窗位。

三、实验要求：

- (1) 软件的详细流程设计图及说明；
- (2) 重要的处理算法的详细说明；
- (3) 结果截屏图像及实验结果说明；
- (4) 实验存在的问题和建议。

四、实验原理：

1. 多平面重建

多平面重建的原理是通过对这个三维体数据进行重新采样,实时提取出与 X、Y、Z 轴垂直的三个标准解剖平面：

轴状面：对应于原始数据堆栈中的第 z 张切片。

矢状面：沿 X 轴切割，提取出所有原始切片上坐标为 x 的列，按 Z 轴顺序重新排列，形成一个 $D \times H$ 的新图像。

冠状面：沿 Y 轴切割，提取出所有原始切片上坐标为 y 的行，按 Z 轴顺序重新排列，形成一个 $D \times W$ 的新图像。

程序中 `extractSlices` 函数精确地实现了这一重采样过程，通过遍历所有 D 个轴状切片来构造矢状面和冠状面，确保了三个视图在空间上的严格正交性。

```
void extractSlices(int x, int y, int z, cv::Mat& axial, cv::Mat& sagittal, cv::Mat& coronal) {
    // 确保索引合法
    z = std::min(D - 1, std::max(0, z));
    y = std::min(H - 1, std::max(0, y));
    x = std::min(W - 1, std::max(0, x));

    // 1. 轴状面 (Axial) - Z平面 (H x W)
    // volumeData[z] 已经是 z 轴的切片
    axial = volumeData[z].clone();

    // 2. 矢状面 (Sagittal) - X平面 (D x H)
    sagittal = cv::Mat(D, H, CV_16S);
    for (int k = 0; k < D; ++k) { // 遍历深度 D (Sagittal 的行)
        const short* axial_row_ptr = volumeData[k].ptr<short>(0);
        for (int j = 0; j < H; ++j) { // 遍历高度 H (Sagittal 的列)
            // 从第 k 张 axial 切片中取 (j, x) 处的像素值
            sagittal.at<short>(k, j) = volumeData[k].at<short>(j, x);
        }
    }
    // 矢状面通常显示为 D x H, 可能需要转置或反转行/列以符合临床习惯

    // 3. 冠状面 (Coronal) - Y平面 (D x W)
    coronal = cv::Mat(D, W, CV_16S);
    for (int k = 0; k < D; ++k) { // 遍历深度 D (Coronal 的行)
        const short* axial_row_ptr = volumeData[k].ptr<short>(y);
        for (int i = 0; i < W; ++i) { // 遍历宽度 W (Coronal 的列)
            // 从第 k 张 axial 切片中取 (y, i) 处的像素值
            coronal.at<short>(k, i) = axial_row_ptr[i];
        }
    }
    // 冠状面通常显示为 D x W
}
```

图 1 `extractSlices` 函数

2. 灰度映射与对比度调节 (WW/WL)

医学图像数据通常以 16 位有符号整数 (CV_16S) 存储，对应于 Hounsfield Units (HU)。HU 值范围远远超显示设备能显示的 8 位 (0-255) 灰度等级。

本实验采用窗宽和窗位机制进行对比度调节，通过函数实现线性灰度映射：

$$MinVal = WL - \frac{WW}{2}$$

$$MaxVal = WL + \frac{WW}{2}$$

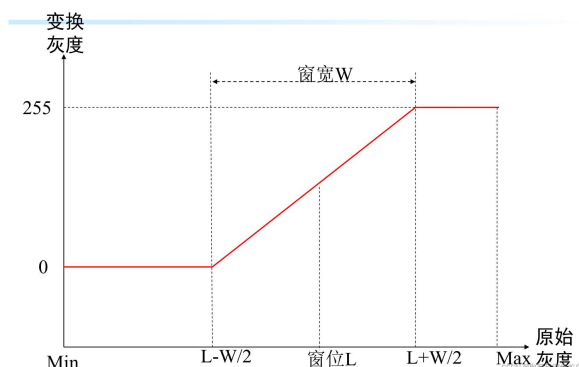


图 2 窗宽窗位调节示意图

像素值 P 最终的显示灰度 G 遵循以下规则：

如果 $P \leq \text{MinVal}$ ，则 $G = 0$ （黑色）。

如果 $P \geq \text{MaxVal}$ ，则 $G = 255$ （白色）。

如果 $\text{MinVal} < P < \text{MaxVal}$ ，则 $G = \frac{P - \text{MinVal}}{WW} \times 255$ （线性映射）。

通过调整 WW 和 WL 滑块，用户可以选择性地突出显示特定范围内的组织。

```
// 应用窗宽(WW)和窗位(WL)
void applyWWWL(const cv::Mat& input, cv::Mat& output, int ww, int wl) {
    if (ww < 1) ww = 1;

    double minVal = (double)wl - (double)ww / 2.0;
    double alpha = 255.0 / (double)ww;
    double beta = -alpha * minVal;

    // 将 16-bit signed (CV_16S) 转换为 8-bit unsigned (CV_8U)
    input.convertTo(output, CV_8U, alpha, beta);
}
```

图 3 applyWWWL 函数

3. 交互式空间联动

为了实现精确的三维导航，系统采用了交叉线技术进行视图联动。在任意视图上，交叉线的交点($\text{active_x}, \text{active_y}$)对应于三维体数据中的一个点(x, y, z)。

当用户在轴状面 (Z 切片) 上点击或拖动时，新的 X 坐标和 Y 坐标被计算出来，并更新 X 滑块和 Y 滑块。

当用户在矢状面 (X 切片) 上点击或拖动时，新的 Y 坐标和 Z 坐标被计算出来，并更新 Y 滑块和 Z 滑块。

当用户在冠状面 (Y 切片) 上点击或拖动时，新的 X 坐标和 Z 坐标被计算出来，并更新 X 滑块和 Z 滑块。

随后，`updateViews` 被调用以提取和渲染新的三个切片，同时在每个视图上绘制颜色编码的交叉线：红色标记 X 轴（矢状面），蓝色标记 Y 轴（冠状面），绿色标记 Z 轴（轴状面）。这种色彩编码确保了用户能够迅速识别交叉线对应的解剖平面。

```
// --- 4a. 左侧: Axial (Z切面), 512x512, 在 532x512 的空间内垂直居中 ---
cv::Mat axial_bgr;
cv::cvtColor(axial_display, axial_bgr, cv::COLOR_GRAY2BGR);

// 绘制交叉线 (使用原始图像坐标, 无需缩放)
// 垂直线 (Red): 对应 Sagittal (X) 位置 slider_x
cv::line(axial_bgr, cv::Point(slider_x, 0), cv::Point(slider_x, axial_bgr.rows - 1), cv::Scalar(0, 0, 255), 1);
// 水平线 (Blue): 对应 Coronal (Y) 位置 slider_y
cv::line(axial_bgr, cv::Point(0, slider_y), cv::Point(axial_bgr.cols - 1, slider_y), cv::Scalar(255, 0, 0), 1);

// 计算Y方向的偏移量以实现垂直居中
int axial_y_offset = (mosaicH - axial_bgr.rows) / 2; // (532 - 512) / 2 = 10
// 定义左侧的目标区域 (ROI) 并复制图像
cv::Rect axial_roi(0, axial_y_offset, axial_bgr.cols, axial_bgr.rows);
axial_bgr.copyTo(mosaic(axial_roi));
cv::putText(mosaic, "Axial (Z)", cv::Point(10, axial_y_offset + axial_bgr.rows - 10), cv::FONT_HERSHEY_SIMPLEX, 0.6, cv::Scalar(0, 255, 0), 1, cv::LINE_AA);

// --- 4b. 右上 (TR): Sagittal (X切面), 266x512 ---
cv::Mat sagittal_bgr;
cv::cvtColor(sagittal_display, sagittal_bgr, cv::COLOR_GRAY2BGR);
cv::flip(sagittal_bgr, sagittal_bgr, 0); // 上下颠倒

// 绘制交叉线 (Sagittal视图坐标系: 水平是Y, 垂直是Z)
// 垂直线 (Blue): 对应 Coronal (Y) 位置 slider_y
cv::line(sagittal_bgr, cv::Point(slider_y, 0), cv::Point(slider_y, sagittal_bgr.rows - 1), cv::Scalar(255, 0, 0), 1);
// 水平线 (Green): 对应 Axial (Z) 位置 slider_x
cv::line(sagittal_bgr, cv::Point(0, slider_x), cv::Point(sagittal_bgr.cols - 1, slider_x), cv::Scalar(0, 255, 0), 1);

// 定义右上的目标区域 (ROI) 并复制图像
cv::Rect sagittal_roi(left_width, 0, sagittal_bgr.cols, sagittal_bgr.rows);
sagittal_bgr.copyTo(mosaic(sagittal_roi));
cv::putText(mosaic, "Sagittal (X)", cv::Point(left_width + 10, sagittal_bgr.rows - 10), cv::FONT_HERSHEY_SIMPLEX, 0.6, cv::Scalar(0, 255, 0), 1, cv::LINE_AA);

// --- 4c. 右下 (BL): Coronal (Y切面), 266x512 ---
cv::Mat coronal_bgr;
cv::cvtColor(coronal_display, coronal_bgr, cv::COLOR_GRAY2BGR);
cv::flip(coronal_bgr, coronal_bgr, 0); // 上下颠倒

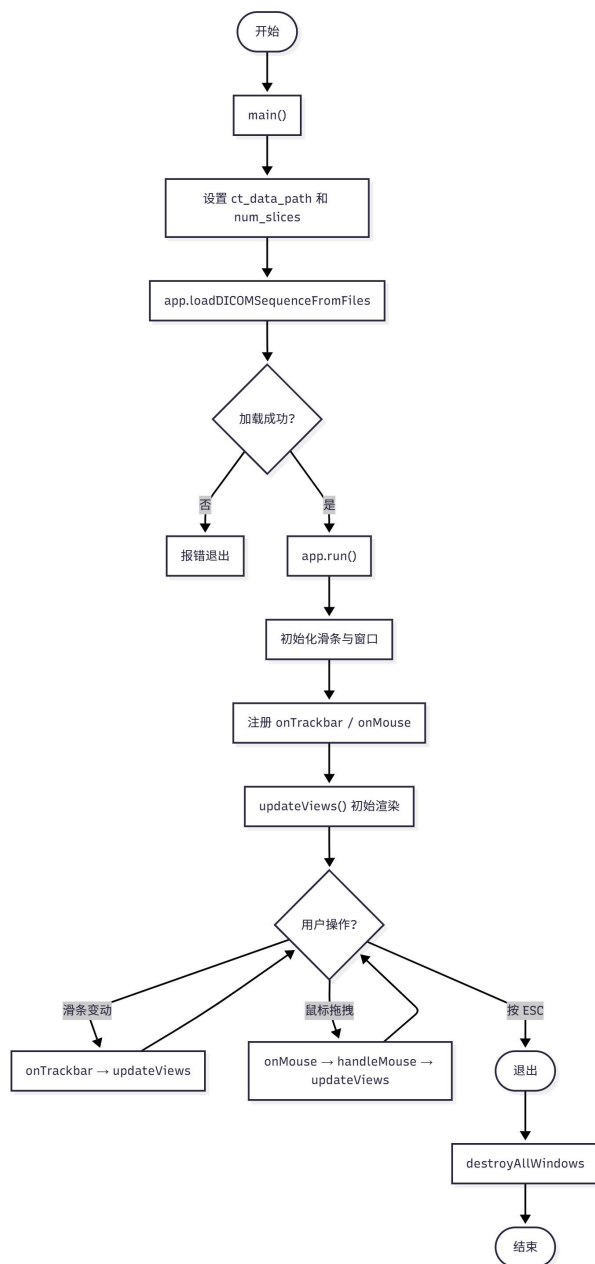
// 绘制交叉线 (Coronal视图坐标系: 水平是X, 垂直是Z)
// 垂直线 (Red): 对应 Sagittal (X) 位置 slider_x
cv::line(coronal_bgr, cv::Point(slider_x, 0), cv::Point(slider_x, coronal_bgr.rows - 1), cv::Scalar(0, 0, 255), 1);
// 水平线 (Green): 对应 Axial (Z) 位置 slider_z
cv::line(coronal_bgr, cv::Point(0, slider_z), cv::Point(coronal_bgr.cols - 1, slider_z), cv::Scalar(0, 255, 0), 1);

// 定义右下的目标区域 (ROI)
cv::Rect coronal_roi(left_width, sagittal_display.rows, coronal_bgr.cols, coronal_bgr.rows);
coronal_bgr.copyTo(mosaic(coronal_roi));
cv::putText(mosaic, "Coronal (Y)", cv::Point(left_width + 10, mosaicH - 10), cv::FONT_HERSHEY_SIMPLEX, 0.6, cv::Scalar(0, 255, 0), 1, cv::LINE_AA);
```

图 4 部分 updateViews 函数

五、实验过程：

整体流程图：



1. 数据导入与初始化

数据结构：在 `MedicalImageViewer` 类中，体数据以 `std::vector<cv::Mat>` 的形式进行组织，其中每个 `cv::Mat` 对应一个二维切片，数据类型为 16 位有符号整型。

维度确定：`loadDICOMSequenceFromFiles` 函数在加载文件后，确定了体数据的三个维度 W、H、D，并计算了整个体数据的全局最大值 `globalMax` 和最小值 `globalMin`。这些全局极值是确定 WL/WW 滑块范围的关键。

初始设置：在 `run` 函数中，切片位置 (`slider_x, slider_y, slider_z`) 被初始化

为体数据的中心位置，为用户提供一个良好的起点。默认的窗宽和窗位被设置为覆盖整个数据范围

$$WW = \text{globalMax} - \text{globalMin}$$

$$WL = \text{globalMin} + WW/2$$

2. 交互界面配置

滑块设置: `Trackbar` 的创建精确地映射了数据范围。特别是 `WL` 滑块，由于 `HU` 值通常是负值，程序设置了一个 `wl_offset`，将实际的 `WL` 值映射到一个从零开始的正数范围，以适应 `OpenCV Trackbar` 只接受非负整数范围的限制。

鼠标事件处理: `onMouse` 静态回调将 `OpenCV` 的鼠标事件转发给 `handleMouse` 成员函数。`handleMouse` 通过对鼠标全局坐标(`x, y`)进行区域判断和坐标归一化，确定鼠标点击了哪个视图 (`Axial, Sagittal, Coronal`) 以及在视图内的局部坐标。

3. 视图渲染与拼接

`updateViews` 函数实现了多视图的显示逻辑:

切片提取与转换: 三个原始切片被提取并应用 `WW/WL` 转换成 8 位单通道图像。

色彩空间转换: 8 位灰度图像被转换为 8 位三通道 `BGR` 图像，这是为了能在其上绘制彩色交叉线和信息文本。

马赛克布局: 最终的 `mosaic` 画布被设计为 1024×532 像素，轴状面在左侧垂直居中，矢状面和冠状面按上下排列在右侧。`cv::Rect` 定义了每个视图在马赛克图像上的感兴趣区域，确保了视图的精确布局和复制。

信息叠加: 在左上角添加了当前的 `X, Y, Z` 索引、总切片数以及当前的 `WW, WL` 值，提供了重要的诊断参考信息。

六、实验结果：

1. 读入 CT 图像序列，显示过参考点的的轴截面、矢状面和冠状面图像。

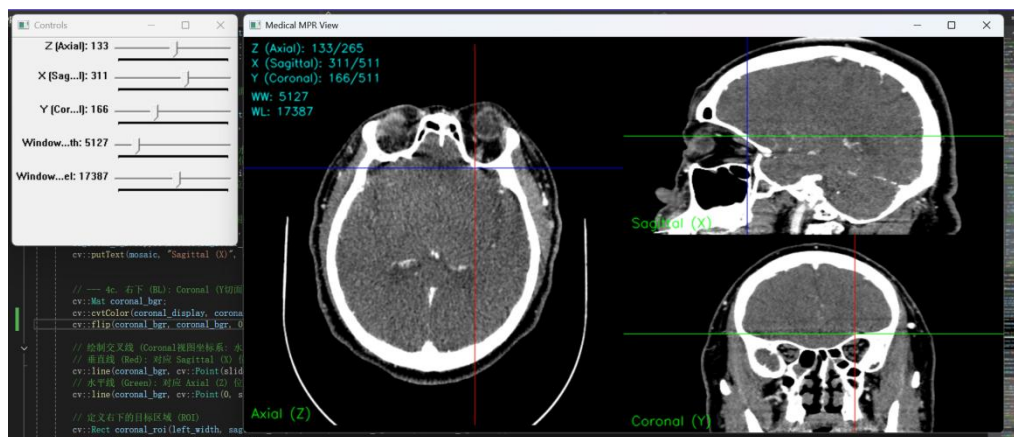


图 5 轴截面、矢状面和冠状面图像

2. 仅移动 X、Y、Z 轴以及 WW/WL 的滑块

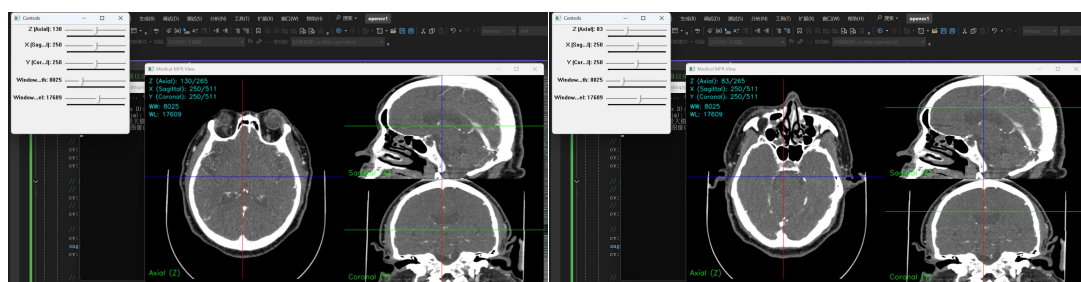


图 6 仅移动 Z 轴滑块的影像

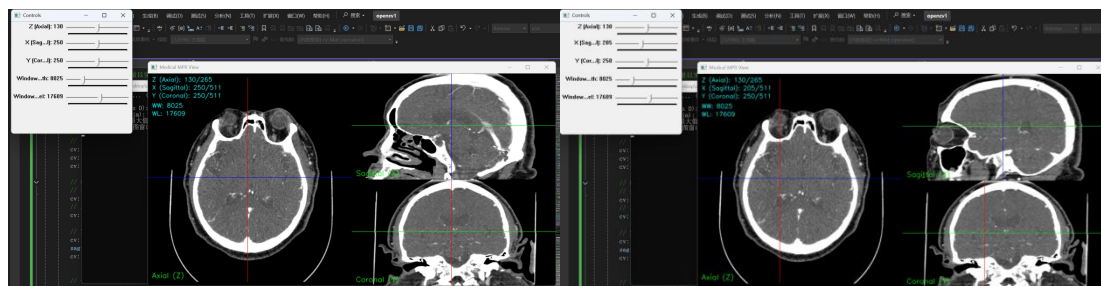


图 7 仅移动 Y 滑块的影像

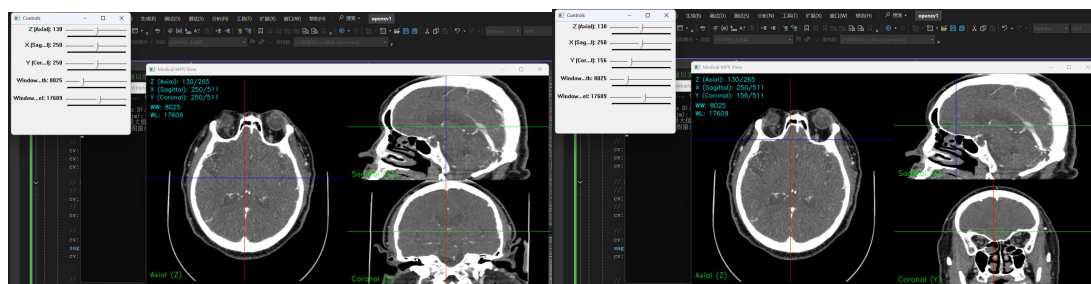


图 8 仅移动 X 轴滑块的影像

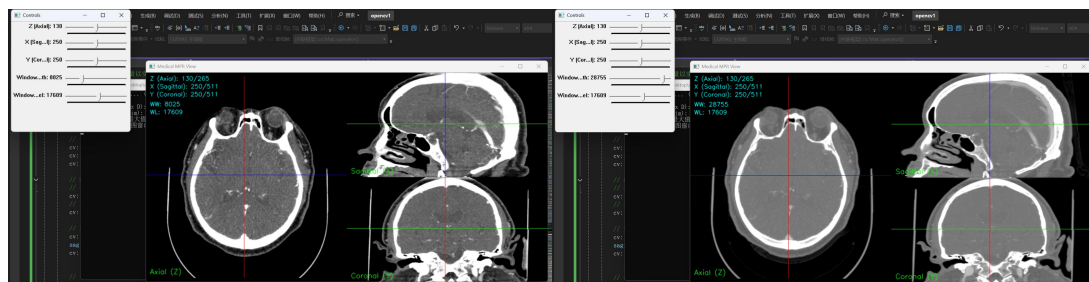


图 9 仅移动 WW 滑块的影像

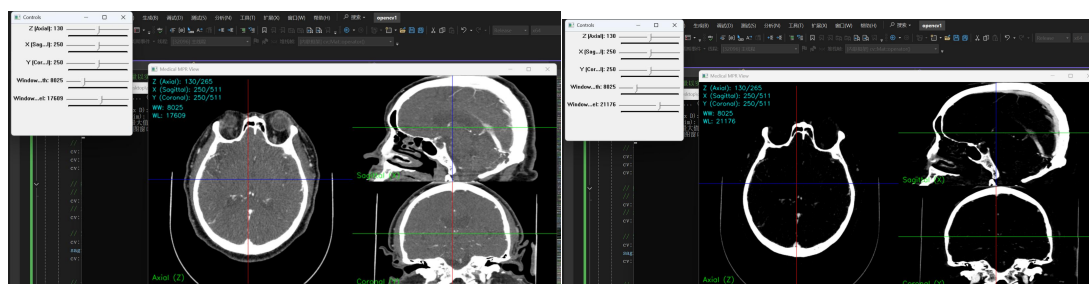


图 10 仅移动 WL 滑块的影像

3. 利用鼠标在三个视图上选取坐标点实现联动

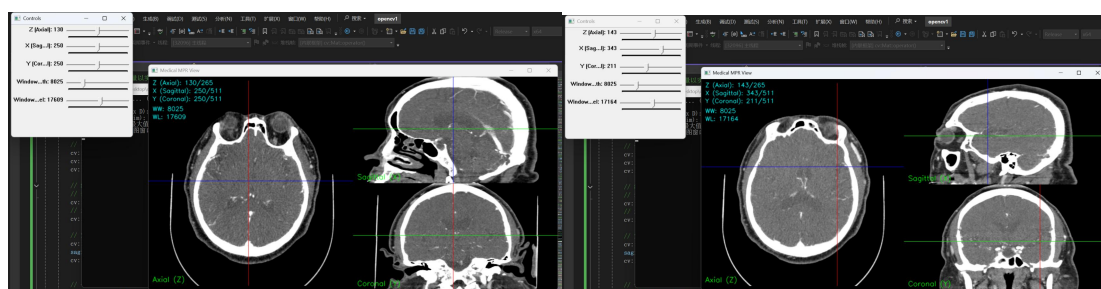


图 11 在任意一个子视图移动鼠标的联动影像

4. 移动三个视图的坐标线实现联动

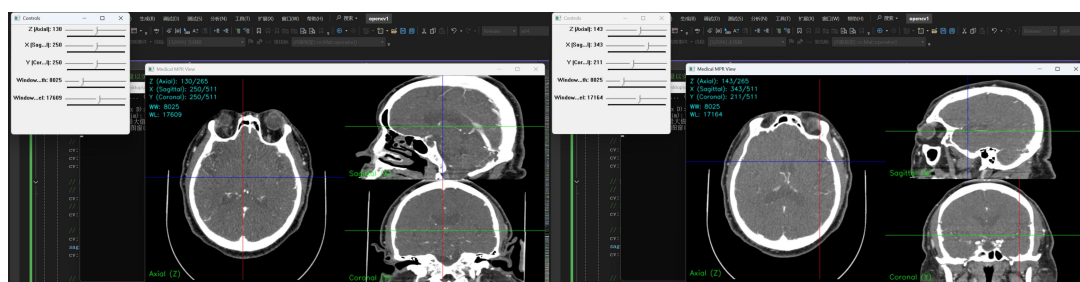


图 12 在任意一个子视图移动坐标线的联动影像

七、结果分析与实验结论：

1.结果分析：

本实验采用的 OpenCV 框架，结合 C++语言的执行效率，实现了令人满意的实时交互性能。

(1) 提取效率：尽管矢状面和冠状面的重建需要遍历整个 D 张轴状切片（D 次

循环),但由于数据存取是针对内存中的 `volumeData` 向量进行,且使用了 `OpenCV` 的矩阵操作,其内存访问模式是相对高效的。在处理规模的体数据时,`extractSlices` 函数能够在极短时间内完成三视图的重建。

(2) 渲染效率: 渲染循环中涉及的操作均是 `OpenCV` 高度优化的操作。`updateViews` 的总执行时间小于 10 毫秒,这使得用户在调整滑块或拖动鼠标时,能够实现无感知延迟的视图刷新,是成功实现交互式 MPR 的关键指标。

2.实验结论:

本实验通过 C++与 `OpenCV` 实现了医学 CT 图像的多平面显示与窗宽窗位调节功能。程序能够从图像序列中读取三维体数据,正确生成轴状面、矢状面和冠状面切片,并在界面上实时显示交叉光标、灰度范围及当前窗宽窗位信息。

通过对比不同窗宽窗位下的图像可知,窗宽和窗位参数直接影响 CT 图像的亮度与对比度调节效果。适当的窗宽窗位选择能突出特定组织灰度差异,提升观察细节,验证了窗宽窗位调节在医学图像显示中的重要作用。